

```

import os
import sys
import threading
import requests

from PySide6.QtCore import Qt, QThread, Signal
from PySide6.QtWidgets import (
    QApplication, QWidget, QLabel, QLineEdit, QPushButton,
    QFileDialog, QProgressBar, QMessageBox, QTextEdit,
    QVBoxLayout, QHBoxLayout, QFrame
)

```

این بخش اول، کتابخانه‌های لازم را وارد می‌کند.

- `os` و `sys`: برای کارهای سیستمی مثل مدیریت فایل‌ها و مسیرها.
- `threading`: برای اجرای کارهای سنگین (مثل دانلود) در پس‌زمینه تا رابط کاربری قفل نشود.
- `requests`: کتابخانه محبوب برای درخواست‌های HTTP (یعنی دانلود فایل از اینترنت).
- `PySide6`: کتابخانه اصلی برای ساخت رابط کاربری گرافیکی (GUI) در پایتون. قسمت‌های مختلفش برای ویجت‌ها، سیگنال‌ها، نخ‌ها و ... استفاده می‌شود.

کلاس DownloadWorker (نخ دانلود)

```

class DownloadWorker(QThread):
    progress = Signal(int)
    status = Signal(str)
    finished_ok = Signal(str)
    finished_fail = Signal(str)

    def __init__(self, url: str, save_path: str):
        super().__init__()
        self.url = url
        self.save_path = save_path

```

```
self._cancel_event = threading.Event()
```

```
def cancel(self):
```

```
    self._cancel_event.set()
```

```
def run(self):
```

```
    # ... (جزئیات دانلود در اینجا) ...
```

این کلاس مسئول انجام خود دانلود در یک نخ جداگانه (Separate Thread) است.

- QThread: پایه این کلاس است که اجرای کد را به صورت موازی ممکن می‌کند.
- Signals: این‌ها مثل پیام‌هایی هستند که نخ دانلود می‌تواند به رابط کاربری اصلی بفرستد (مثلاً پیشرفت دانلود، وضعیت، موفقیت یا شکست).
- __init__: آدرس فایل (url) و مسیر ذخیره (save_path) را می‌گیرد. cancel_event هم برای مدیریت دکمه لغو است.
- cancel(): این متد وقتی دکمه لغو زده می‌شود، cancel_event را فعال می‌کند تا نخ دانلود بفهمد باید متوقف شود.
- run(): این متد اصلی است که QThread وقتی شروع به کار می‌کند، اجراش می‌کند. تمام منطق دانلود اینجا قرار دارد.

منطق run () داخل DownloadWorker

```
def run(self):
```

```
    temp_path = self.save_path + ".part" # فایل موقت
```

```
    try:
```

```
        self.status.emit("...در حال اتصال به سرور")
```

```
        os.makedirs(os.path.dirname(self.save_path) or ".", exist_ok=True) # ساخت پوشه
```

```
        if os.path.exists(temp_path): # اگر فایل ناقص قبلی بود، پاکش کن
```

```
            try: os.remove(temp_path)
```

```
            except Exception: pass
```

with requests.get(self.url, stream=True, timeout=30) as r: # با استریمینگ GET درخواست

r.raise_for_status() # بررسی خطاها

total = r.headers.get("content-length") # گرفتن حجم کل فایل

total = int(total) if total and total.isdigit() else 0

downloaded = 0

chunk_size = 1024 * 64 # اندازه هر بسته دانلود (64KB)

self.status.emit("...دانلود شروع شد")

with open(temp_path, "wb") as f: # باز کردن فایل موقت برای نوشتن

for chunk in r.iter_content(chunk_size=chunk_size): # خواندن محتوا تکه تکه

if self._cancel_event.is_set(): # اگر کاربر لغو کرد

... (پاک کردن فایل موقت و ارسال سیگنال لغو)

return

if not chunk: continue # اگر بسته خالی بود، رد شو

f.write(chunk) # نوشتن بسته در فایل

downloaded += len(chunk) # به‌روزرسانی حجم دانلود شده

if total > 0: # اگر حجم کل مشخص بود

percent = int(downloaded * 100 / total)

self.progress.emit(max(0, min(100, percent))) # UI ارسال پیشرفت به

if total == 0: self.progress.emit(100) # اگر حجم کل صفر بود، یعنی 100%

os.replace(temp_path, self.save_path) # تغییر نام فایل موقت به نام اصلی (کامل شد!)

self.finished_ok.emit(self.save_path) # ارسال سیگنال موفقیت

```

except Exception as e: # اگر خطایی رخ داد
    ... (پاک کردن فایل موقت و ارسال سیگنال خطا) ...
    self.finished_fail.emit(f"خطا: {e}")

```

این بخش، قلب دانلودر است:

1. **فایل موقت (part.):** یک فایل موقت با پسوند part ایجاد می‌شود. اگر دانلود قطع شد یا لغو شد، این فایل پاک می‌شود. اگر فایل part از قبل وجود داشت، یعنی دانلود قبلی نیمه‌کاره مانده، پس آن را پاک می‌کنیم.
2. **درخواست GET:** با requests.get، درخواست دانلود فایل فرستاده می‌شود. stream=True یعنی فایل را تکه‌تکه دریافت می‌کنیم، نه یک‌باره در حافظه.
3. **بررسی خطا:** r.raise_for_status() اگر سرور پاسخ خطا داد (مثل 404)، برنامه را متوقف می‌کند.
4. **حجم فایل:** حجم کل فایل از هدر پاسخ گرفته می‌شود.
5. **حلقه دانلود:** فایل تکه‌تکه (chunk) دریافت و در فایل موقت نوشته می‌شود.
6. **بررسی لغو:** در هر مرحله، چک می‌کند که آیا سیگنال لغو دریافت شده است یا نه. اگر شده بود، فایل موقت را پاک کرده و پیام لغو را می‌فرستد.
7. **پیشرفت:** درصد پیشرفت محاسبه و به رابط کاربری ارسال می‌شود.
8. **تغییر نام:** بعد از اتمام موفقیت‌آمیز، فایل موقت (part.) به نام اصلی تغییر نام داده می‌شود.
9. **مدیریت خطا:** اگر هر مشکلی پیش آمد (مثل قطع اینترنت، خطای سرور و ...)، فایل موقت پاک شده و پیام خطا به رابط کاربری فرستاده می‌شود.

کلاس DownloaderApp (رابط کاربری اصلی)

```
class DownloaderApp(QWidget):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.worker = None # نگهداری نخ دانلود
```

```
        self.save_path = "" # مسیر ذخیره فایل
```

```
        self.setup_ui() # ساخت ظاهر برنامه
```

```
    def setup_ui(self):
```

```
        ... (تنظیمات ظاهر پنجره، فونت‌ها، رنگ‌ها و ...) ...
```

```

self.url_edit = QLineEdit() # فیلد ورود URL
self.path_edit = QLineEdit() # فیلد نمایش مسیر ذخیره
self.browse_btn = QPushButton("انتخاب مسیر") # دکمه انتخاب مسیر

self.status_label = QLabel("آماده برای دانلود") # نمایش وضعیت فعلی
self.progress = QProgressBar() # نوار پیشرفت دانلود

self.start_btn = QPushButton("دانلود") # دکمه شروع دانلود
self.cancel_btn = QPushButton("لغو") # دکمه لغو دانلود

self.log = QTextEdit() # ناحیه نمایش لاگ و پیام‌ها
# ... (ها Layout چیدمان ویجت‌ها با) ...

```

این کلاس، ظاهر و تعاملات برنامه را مدیریت می‌کند.

- `__init__`: پنجره اصلی را می‌سازد و متد `setup_ui` را صدا می‌زند.
- `setup_ui()`: تمام ویجت‌ها (فیلدها، دکمه‌ها، نوار پیشرفت، ناحیه لاگ) را ایجاد کرده، ظاهرشان را تنظیم می‌کند (با استفاده از `setStyleSheet`) و با استفاده از `Layout` ها آن‌ها را در پنجره مرتب می‌کند.
- `add_log(text)`: یک پیام به ناحیه لاگ اضافه می‌کند.
- `guess_filename(url)`: سعی می‌کند نام فایل را از انتهای `URL` حدس بزند.
- `pick_path()`: وقتی دکمه “انتخاب مسیر” زده می‌شود، پنجره انتخاب فایل را باز می‌کند تا کاربر محل ذخیره را انتخاب کند.
- `start_download()`:
- ورودی‌ها (`URL` و مسیر) را چک می‌کند.
- یک نمونه جدید از `DownloadWorker` می‌سازد.
- سیگنال‌های `DownloadWorker` را به متدهای رابط کاربری وصل می‌کند (مثلاً `progress` به `self.progress.setValue`).
- دکمه‌ها را فعال/غیرفعال می‌کند.
- نخ دانلود را شروع می‌کند (`self.worker.start()`).
- `cancel_download()`: متد `cancel()` را روی نخ دانلود صدا می‌زند.
- `on_status(text)`: وضعیت فعلی را در `status_label` نمایش می‌دهد و به لاگ اضافه می‌کند.
- `on_success(path)`: وقتی دانلود با موفقیت تمام شد، پیام موفقیت نمایش می‌دهد، دکمه‌ها را تنظیم می‌کند و یک پیام `QMessageBox` نشان می‌دهد.
- `on_fail(msg)`: وقتی دانلود با خطا تمام شد، پیام خطا را نمایش می‌دهد، دکمه‌ها را تنظیم می‌کند و یک پیام `QMessageBox` نشان می‌دهد.

تابع `main()`

```
def main():  
    app = QApplication(sys.argv) # ایجاد نمونه اپلیکیشن Qt  
    win = DownloaderApp() # ساخت پنجره اصلی برنامه  
    win.show() # نمایش پنجره  
    sys.exit(app.exec()) # ... برای پاسخ به کلیک‌ها و Qt شروع حلقه رویداد
```

```
if __name__ == "__main__":  
    main() # اگر فایل مستقیماً اجرا شود main اجرای تابع #
```

این بخش، نقطه شروع برنامه است. یک نمونه `QApplication` می‌سازد، پنجره اصلی (`DownloaderApp`) را ایجاد و نمایش می‌دهد و برنامه را در حلقه رویداد `exec()` قرار می‌دهد تا منتظر دستورات کاربر بماند.

هدف کلی این نسخه

این نسخه UI تمیزتر و استفاده راحت‌تری دارد. بخش دانلود در یک نخ جداگانه اجرا می‌شود که باعث می‌شود رابط کاربری حتی در حین دانلود هم پاسخگو بماند. قابلیت لغو دانلود هم کاملاً کار می‌کند و فایل ناقص را پاک می‌کند.